Codyssey Mission AI 활용법

AI Native 입학연수과정 미션 평가를 진행하며 활용한 AI 프롬프트

2026-11-10 임예성

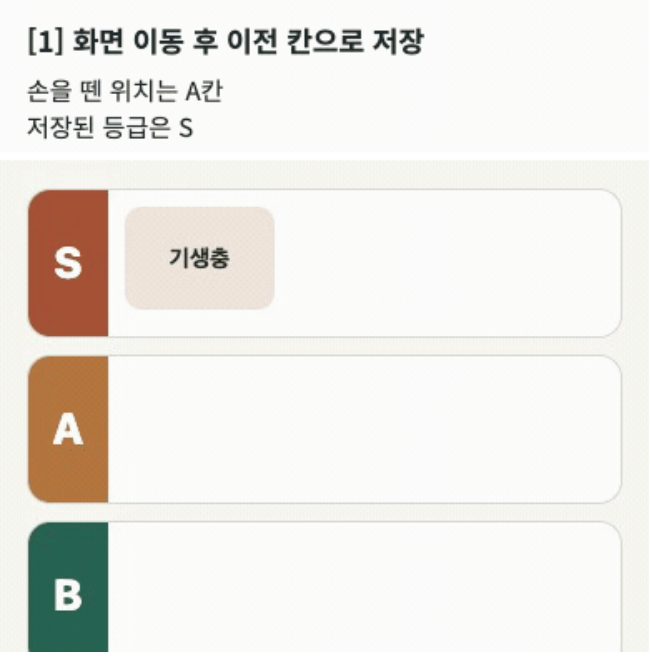
코디세이 입학연수임예성 2026. 11. 10.

GIF 애니메이션

저장 위치

주황색 손가락이 멈춘 뒤 초록 테두리가 어느 칸을 따라가는지 본다.

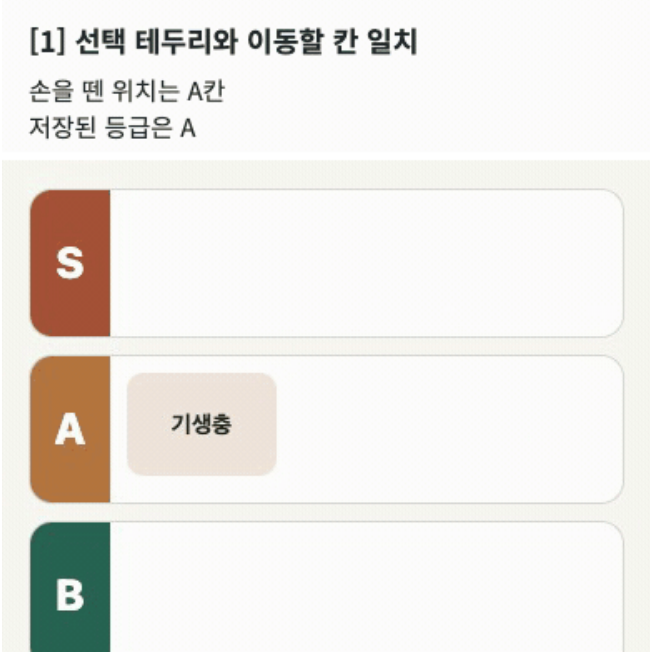
수정 전 S에 저장



손끝은 A에 있었지만 초록 테두리는 S에 남았다. 손을 떼자 〈기생충〉이 S에 저장됐다.

수정 전 원본 GIF

수정 후 A에 저장



화면이 내려가자 초록 테두리가 A로 옮겨 갔다. 손을 떼자 〈기생충〉이 A에 저장됐다.

수정 후 원본 GIF

수정 전후 코드 펼쳐 보기

번호 옆의 굵은 선은 바뀐 줄을 가리킨다.

1단계. 손을 뗐을 때 저장되는 등급

손을 뗀 좌표를 읽고, 그 아래 칸을 확인한 뒤 등급을 저장한다.

수정 전

```
319     document.addEventListener('touchend',event => {
320       if (!touchDrag || ![...event.changedTouches].some(t => t.identifier === touchDrag.id)) return;
321       if (touchDrag.step === 'drag' && event.cancelable) event.preventDefault();   // 놓은 뒤 클릭(선택)이 덧붙지 않게
322       endTouchDrag(true);
323     },{ passive:false });
```

1-4/영화_기능추가2_모바일드래그.html · 319–323행

수정 후

```
319     document.addEventListener('touchend',event => {
320       const touch = touchDrag && [...event.changedTouches].find(t => t.identifier === touchDrag.id);
321       if (!touch) return;
322       touchDrag.x = touch.clientX;
323       touchDrag.y = touch.clientY;
324       if (touchDrag.step === 'drag') {
325         if (event.cancelable) event.preventDefault();   // 놓은 뒤 클릭(선택)이 덧붙지 않게
326         markTouchZone();
327       }
328       endTouchDrag(true);
329     },{ passive:false });
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 319–329행

2단계. 손을 떼기 전에 보이는 테두리

화면이 따로 움직이면 손가락의 위치는 그대로여도 그 아래 칸이 바뀐다. 수정 후에는 드래그 중 매 프레임 그 칸을 다시 찾는다.

수정 전

```
292       if (push) { window.scrollBy(0,Math.max(-18,Math.min(18,Math.round(push / 3)))); markTouchZone(); }
293       touchDrag.frame = requestAnimationFrame(autoScroll);
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 292–293행

수정 후

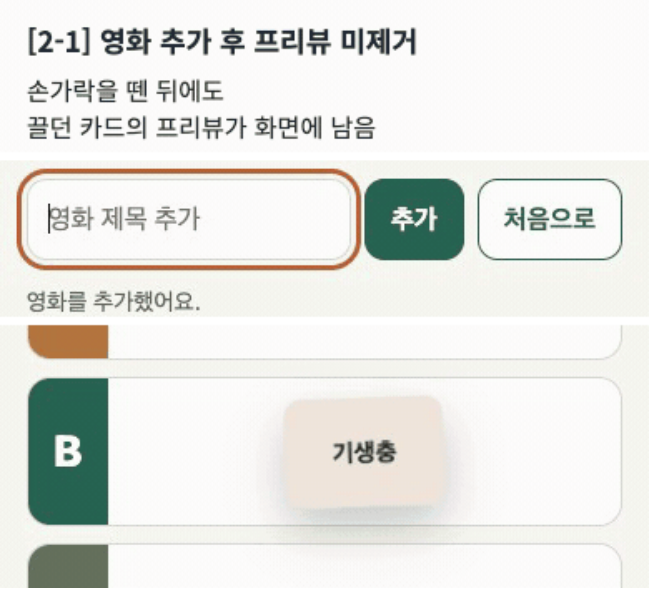
```
292         if (push) {
293           window.scrollBy(0,Math.max(-18,Math.min(18,Math.round(push / 3))));
294         }
295         // 다른 조작으로 화면이 움직여도 손가락 아래 칸을 계속 표시
296         markTouchZone();
297         touchDrag.frame = requestAnimationFrame(autoScroll);
```

1-4/영화_기능추가2_모바일드래그_수정2.html · 292-297행

영화 추가

카드를 든 채 영화를 추가한다. 목록이 바뀐 다음에도 손 아래를 본다.

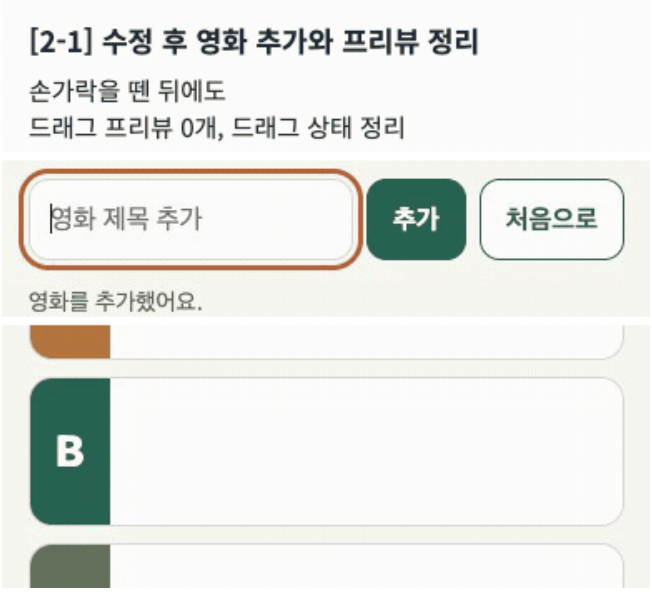
수정 전 카드 모양이 남음



영화를 추가해 목록을 다시 그린 뒤에도 손에 든 카드 모양이 남았다.

수정 전 원본 GIF

수정 후 카드 모양이 사라짐



목록을 다시 그리기 전에 드래그를 끝내자 손에 든 카드 모양이 사라졌다.

수정 후 원본 GIF

수정 전후 코드 펼쳐 보기

번호 옆의 굵은 선은 바뀐 줄을 가리킨다.

영화 추가와 초기화는 서로 다른 곳에서 목록을 다시 그린다. 두 경우 모두 같은 수정에서 드래그를 정리했다.

추가 폼이 목록을 다시 그리는 곳

```
209       state.items.push({ id: 'm' + state.nextId++, title, tier: 'pool' });
210       save(); render(); input.value = ''; showMessage('영화를 추가했어요.'); input.focus();
211     });
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 209–211행

목록을 다시 그리는 첫 줄

목록을 다시 그리기 전에 진행 중인 드래그를 끝낸다. false를 넘기면 카드를 새 등급에 놓지 않고 복사해 띄운 카드와 테두리를 지운다.

수정 전

```
186     function render() {
187       const tiers = document.querySelector('#tiers'); tiers.replaceChildren();
188       for (const tier of LEVELS.filter(value ⇒ value ≠= 'pool')) {
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 186–188행

수정 후

```
186     function render() {
187       endTouchDrag(false);
188       const tiers = document.querySelector('#tiers'); tiers.replaceChildren();
189       for (const tier of LEVELS.filter(value ⇒ value ≠= 'pool')) {
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 186–189행

▼ 정리 함수의 실제 코드

드래그 정리 함수

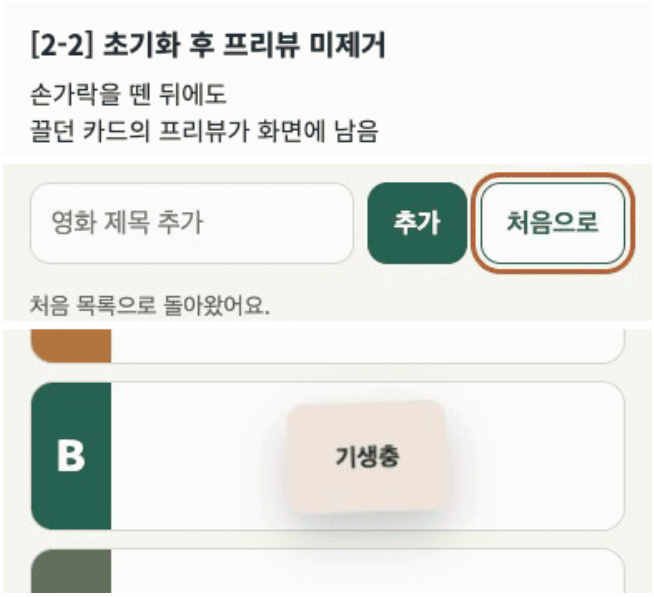
```
291     function endTouchDrag(drop) {
292       const drag = touchDrag; touchDrag = null;
293       if (!drag) return;
294       clearTimeout(drag.timer); cancelAnimationFrame(drag.frame);
295       drag.card.classList.remove('touch-hold','touch-lift','touch-source'); drag.card.drag
    gable = true;
296       drag.zone?.mark.classList.remove('over'); drag.ghost?.remove();
297       const item = state.items.find(item ⇒ item.id ≡≡ drag.card.dataset.id);
298       if (drop && drag.step ≡≡ 'drag' && drag.zone && item && item.tier ≠= drag.zone.tie
    r) move(item.id,drag.zone.tier);
299     }
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 291–299행

초기화

카드를 든 채 '처음으로'를 누른다. 초기 목록 위에 남은 것이 있는지 본다.

수정 전 초기화 뒤에도 남음



'처음으로'를 눌러 초기 목록을 불러왔지만 끌던 카드 모양은 화면에 남았다.

수정 전 원본 GIF

수정 후 초기화할 때 사라짐



초기 목록을 그리기 전에 끌기를 끝내자 떠 있던 카드 모양도 함께 사라졌다.

수정 후 원본 GIF

수정 전후 코드 펼쳐 보기

번호 옆의 굵은 선은 바뀐 줄을 가리킨다.

영화 추가와 초기화는 서로 다른 곳에서 목록을 다시 그린다. 두 경우 모두 같은 수정에서 드래그를 정리했다.

처음으로 버튼이 목록을 다시 그리는 곳

```
212     document.querySelector('#reset').addEventListener('click',() ⇒ {
213       state = initial(); selected = null; document.querySelector('#title').value = ''; sav
    e(); render(); showMessage('처음 목록으로 돌아왔어요.');
214     });
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 212–214행

목록을 다시 그리는 첫 줄

목록을 다시 그리기 전에 진행 중인 드래그를 끝낸다. false를 넘기면 카드를 새 등급에 놓지 않고 복사해 띄운 카드와 테두리를 지운다.

수정 전

```
186     function render() {
187       const tiers = document.querySelector('#tiers'); tiers.replaceChildren();
188       for (const tier of LEVELS.filter(value ⇒ value ≠= 'pool')) {
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 186–188행

수정 후

```
186     function render() {
187       endTouchDrag(false);
188       const tiers = document.querySelector('#tiers'); tiers.replaceChildren();
189       for (const tier of LEVELS.filter(value ⇒ value ≠= 'pool')) {
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 186–189행

▼ 정리 함수의 실제 코드

드래그 정리 함수

```
291     function endTouchDrag(drop) {
292       const drag = touchDrag; touchDrag = null;
293       if (!drag) return;
294       clearTimeout(drag.timer); cancelAnimationFrame(drag.frame);
295       drag.card.classList.remove('touch-hold','touch-lift','touch-source'); drag.card.drag
    gable = true;
296       drag.zone?.mark.classList.remove('over'); drag.ghost?.remove();
297       const item = state.items.find(item ⇒ item.id ≡≡ drag.card.dataset.id);
298       if (drop && drag.step ≡≡ 'drag' && drag.zone && item && item.tier ≠= drag.zone.tie
    r) move(item.id,drag.zone.tier);
299     }
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 291–299행

2배 확대

화면을 두 배로 확대한 뒤 창 아래에 카드를 댄다. 등급 칸이 올라오는지 본다.

수정 전 목록이 멈춤



2배로 확대한 창 아래에 카드를 대도 목록이 내려가지 않았다.

수정 전 원본 GIF

수정 후 목록이 내려감



실제로 보이는 창 아래에 카드를 대자 목록이 내려가며 다음 칸이 보였다.

수정 후 원본 GIF

수정 전후 코드 펼쳐 보기

번호 옆의 굵은 선은 바뀐 줄을 가리킨다.

자동 스크롤을 시작하는 경계

두 배로 확대하면 지금 보이는 화면의 아래끝이 달라진다. 확대된 화면의 시작점과 높이를 읽어서 위아래 경계를 다시 잡는다.

수정 전

```
284     function autoScroll() {
285       if (touchDrag?.step !== 'drag') return;
286       const y = touchDrag.y, bottom = window.innerHeight - EDGE;
287       const push = y < EDGE ? y - EDGE : y > bottom ? y - bottom : 0;
288       if (push) { window.scrollBy(0,Math.max(-18,Math.min(18,Math.round(push / 3)))); mark
    TouchZone(); }
289       touchDrag.frame = requestAnimationFrame(autoScroll);
290     }
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 284–290행

수정 후

```
284     function autoScroll() {
285       if (touchDrag?.step !== 'drag') return;
286       const viewportTop = window.visualViewport?.offsetTop ?? 0;
287       const viewportHeight = window.visualViewport?.height ?? window.innerHeight;
288       const top = viewportTop + EDGE;
289       const bottom = viewportTop + viewportHeight - EDGE;
290       const y = touchDrag.y;
291       const push = y < top ? y - top : y > bottom ? y - bottom : 0;
292       if (push) { window.scrollBy(0,Math.max(-18,Math.min(18,Math.round(push / 3)))); mark
    TouchZone(); }
293       touchDrag.frame = requestAnimationFrame(autoScroll);
294     }
```

1-4/영화_기능추가2_모바일드래그_수정1.html · 284–294행

같은 손동작

화면 크기와 시작 상태가 달라지면 같은 손동작을 해도 손끝이 다른 칸에 닿는다. 두 파일의 조건을 맞추고 손가락이 닿은 자리를 표시해 달라고 했다.

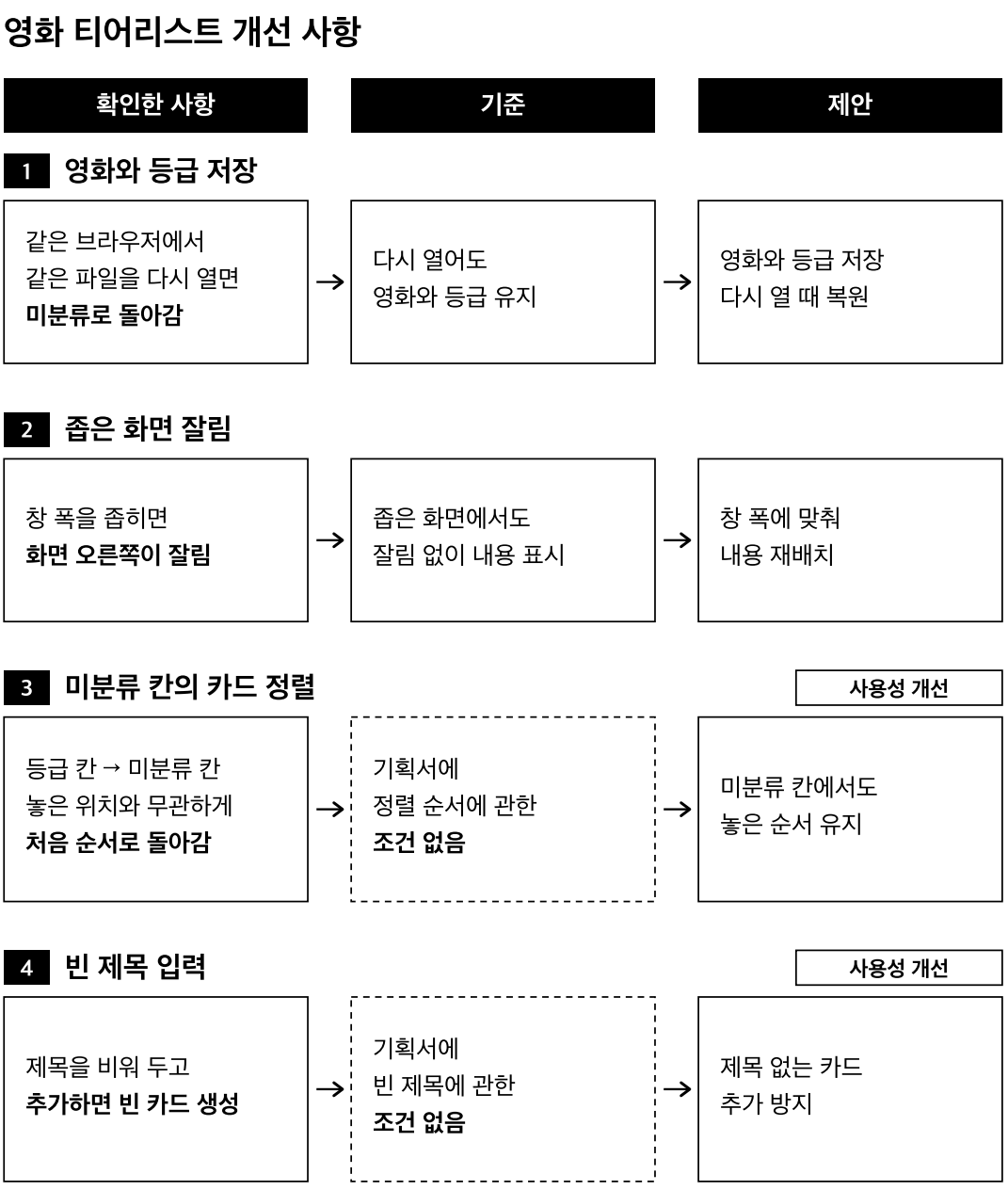
Prompt 1. 재현 영상

> 수정 전 HTML과 수정 후 HTML에서 같은 문제를 비교하는 Before/After GIF 애니메이션 한 쌍을 만들어 줘.
> 두 파일에서 화면 크기, 시작 상태, 같은 조작 순서를 맞추고 단계 자막을 넣어 줘.
> 마우스나 손가락이 닿는 곳에는 터치 지점 표시를 넣어 줘.
> 마지막 결과가 달라지는 지점이 보이도록 잠시 멈춰 보여 주고, 재현하지 못한 것은 따로 적어 줘.

피드백 도식

기획 조건과 사용성 제안

영화 카드를 미분류 칸에 돌려놓자 처음 순서로 돌아왔다. 동료에게 놓은 순서대로 두면 좋겠다고 썼다. 그런데 그 동료의 기획서에는 미분류 칸의 정렬 기준이 없었다.



점선은 기획서에 기준이 없던 두 항목을 가리킨다.

'카드 정렬' 행을 따라가면 가운데 칸의 선이 끊겨 있다. '빈 제목 입력'도 마찬가지다. 두 제안을 사용성 개선으로 적은 이유다.

▼ 원문 표 읽기

항목	확인한 사항	기준	제안
영화와 등급 저장	같은 브라우저에서 같은 파일을 다시 열면 미분류로 돌아감	영화와 등급 유지	저장 후 다시 열 때 복원

항목	확인한 사항	기준	제안
좁은 화면 잘림	폭을 좁히면 오른쪽이 잘림	좁은 화면에서도 내용 표시	창 폭에 맞춰 재배치
카드 정렬	'아직 안 정한 영화'로 되돌리면 놓은 위치와 무관하게 처음 순서로 복귀	기획서에 정렬 순서에 관한 조건 없음	미분류 칸에서도 놓은 순서 유지 (사용성 개선)
빈 제목 입력	제목을 비워 두고 추가하면 빈 카드가 생김	기획서에 빈 제목에 관한 조건 없음	제목 없는 카드 추가 방지 (사용성 개선)

피드백 원문 원본 SVG

피드백 도식의 제작 조건

Prompt 2. 피드백 도식

> 첨부한 내 글을 바탕으로 피드백 표의 네 항목을 도식으로 만들어 줘.
> 각 항목이 왼쪽에서 오른쪽으로 '확인한 사항 → 기준 → 제안' 순서로 읽히게 해 줘.
> 항목 이름과 문장은 그대로 두고, 기획서에 조건이 없는 기준은 점선 상자로 표시해 줘.
> 흰 바탕과 검은 글자, 선으로 그려 줘. 없는 조건이나 새 주장을 보태지 말아 줘.

레이아웃 레퍼런스

목차와 본문 사이의 여백

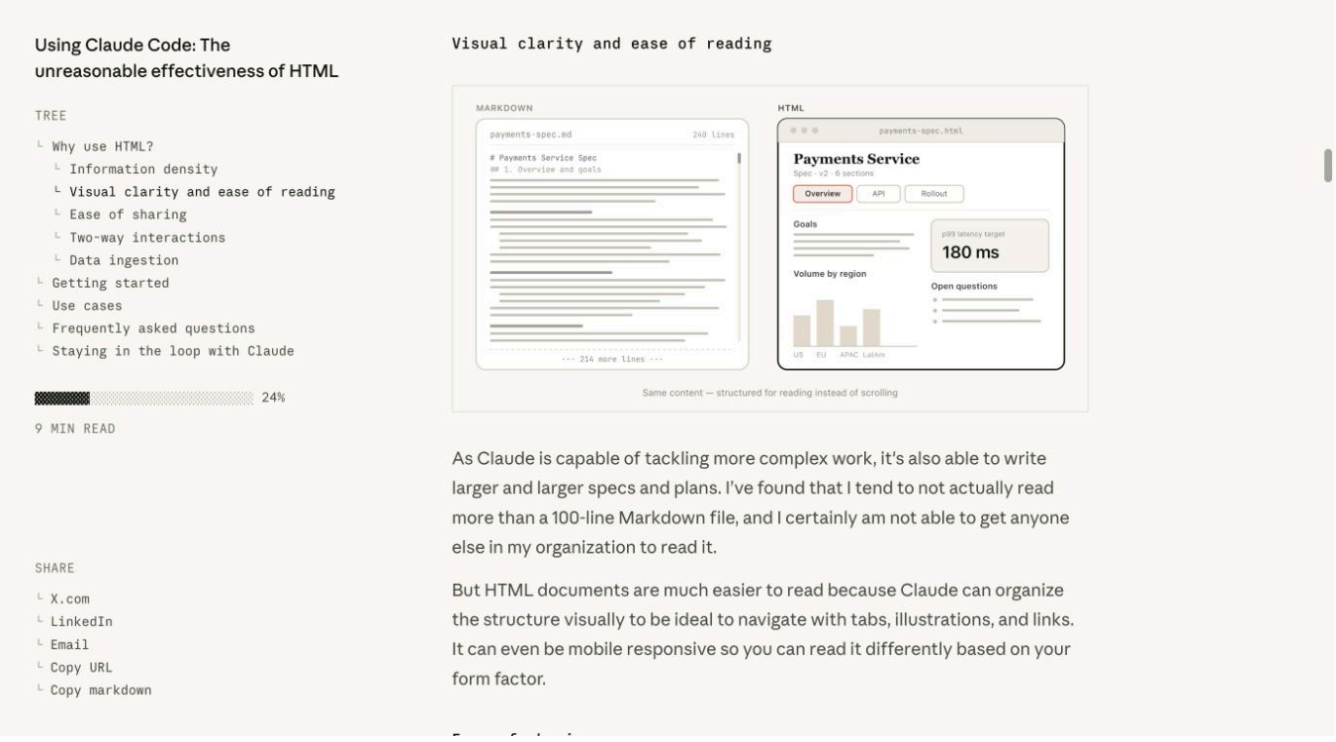
영상을 전공한 입장에서 '화면이 깔끔하다'는 표현은 추상적이다. AI에게 페이지 제작을 요청할 때는 목차를 어디에 두고 본문을 얼마나 넓게 쓸지 구체적으로 설명해야 했다.

OpenAI 크리에이티브 팀의 인터뷰 페이지에서는 글을 내려도 목차가 왼쪽에 고정돼 있다. 목차와 본문 사이에는 테두리 없이 넓은 여백이 있다. 이 구성이 마음에 들어 발표 페이지에도 같은 방식으로 목차를 배치해 달라고 했다.

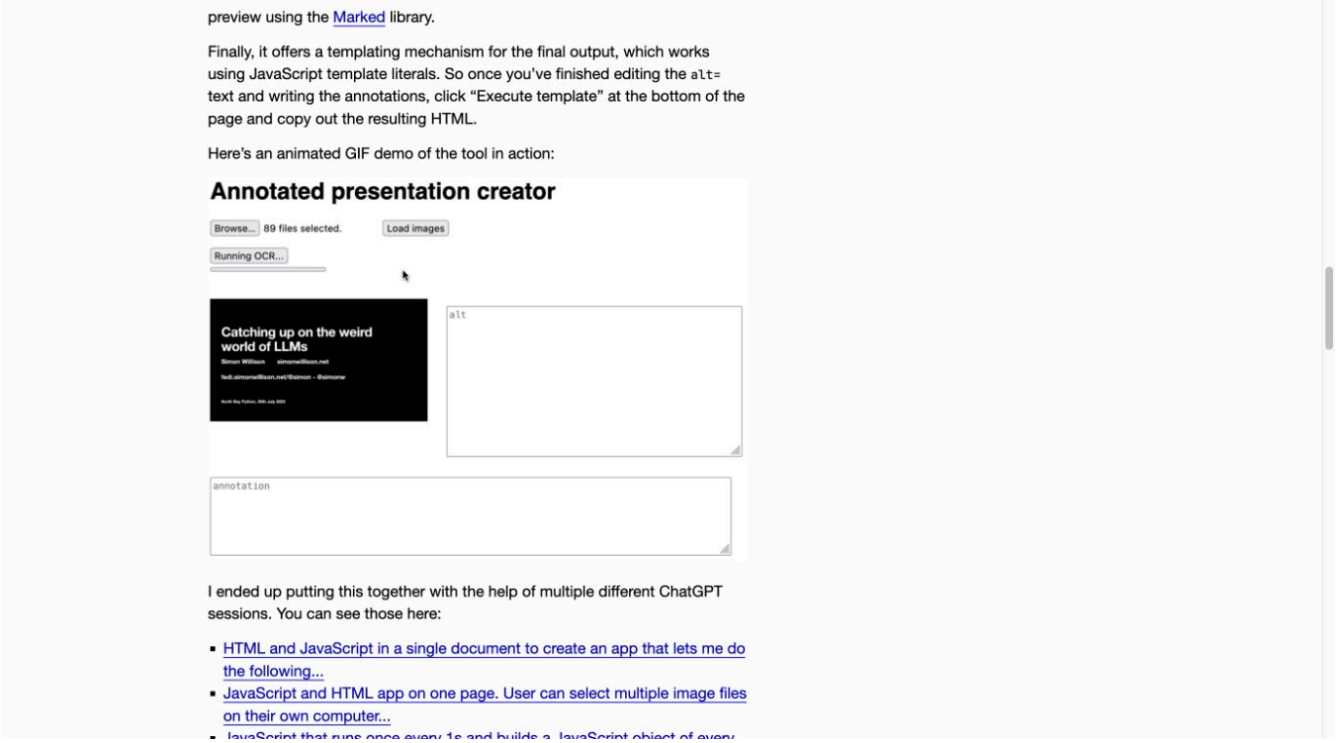
처음 조사한 여섯 페이지에서는 이미지의 너비와 설명이 붙는 위치를 비교했다. 각 페이지에서 참고한 부분을 한 장씩 캡처하고, 그 아래에 좋았던 점을 한 줄로 적었다.



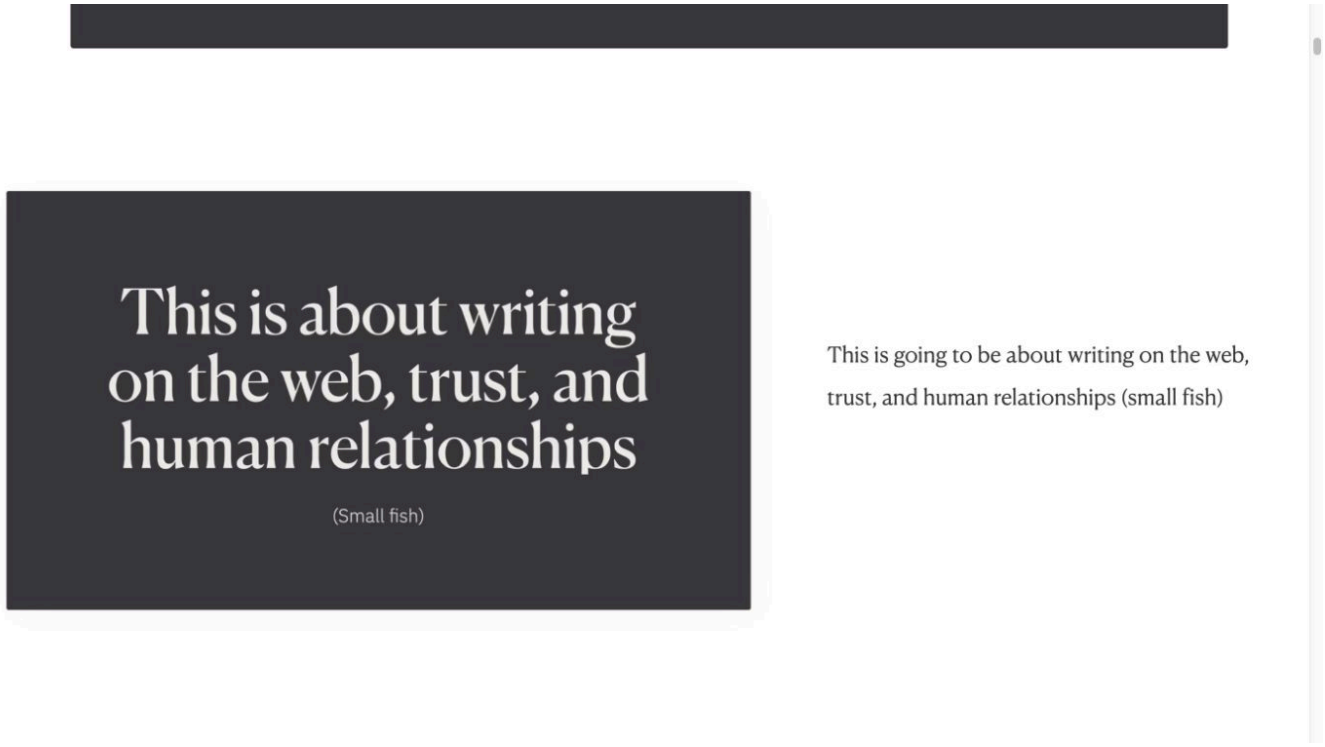
OpenAI 원문 목차와 본문 사이를 흰 여백만으로 나눈 게 좋았음.



Thariq Shihipar 원문 같은 내용을 두 화면에 나란히 놓으니 바뀐 곳이 바로 보임.



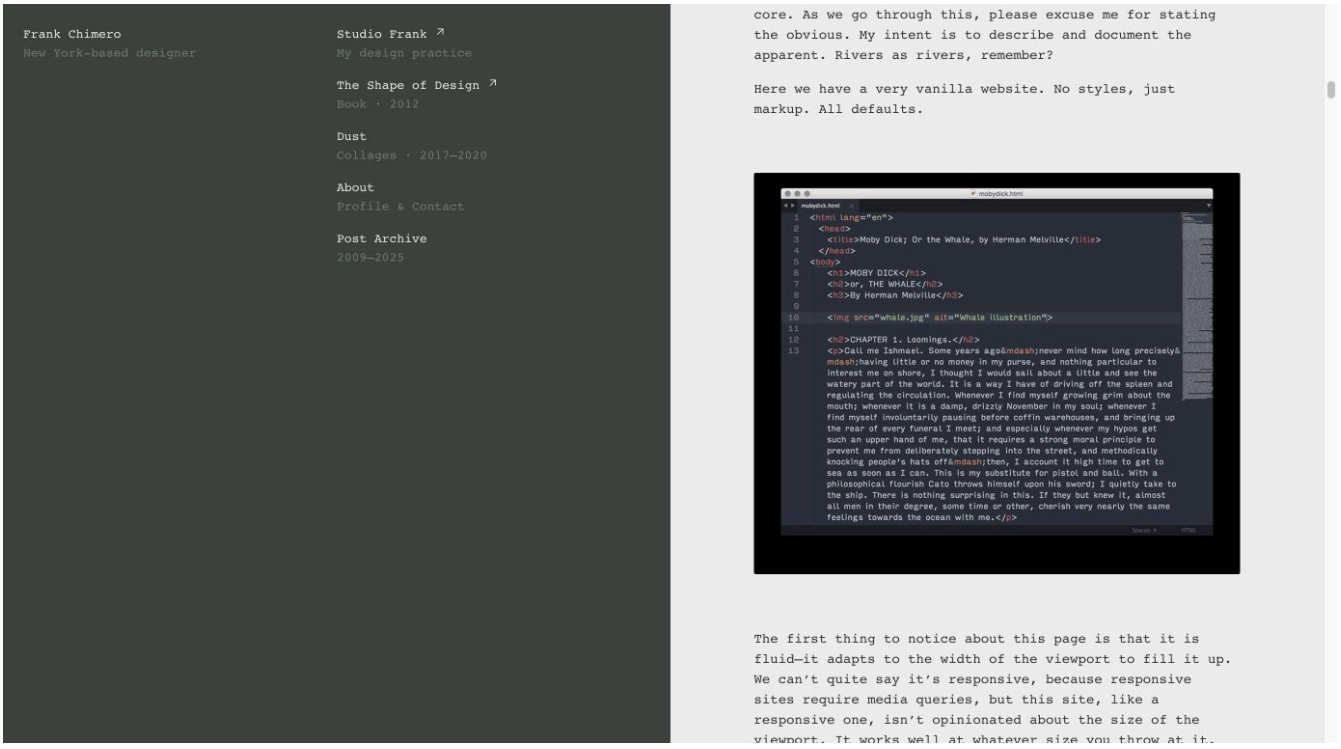
Simon Willison 원문 설명을 읽고 눈을 조금 내리면 방금 말한 도구가 바로 움직임.



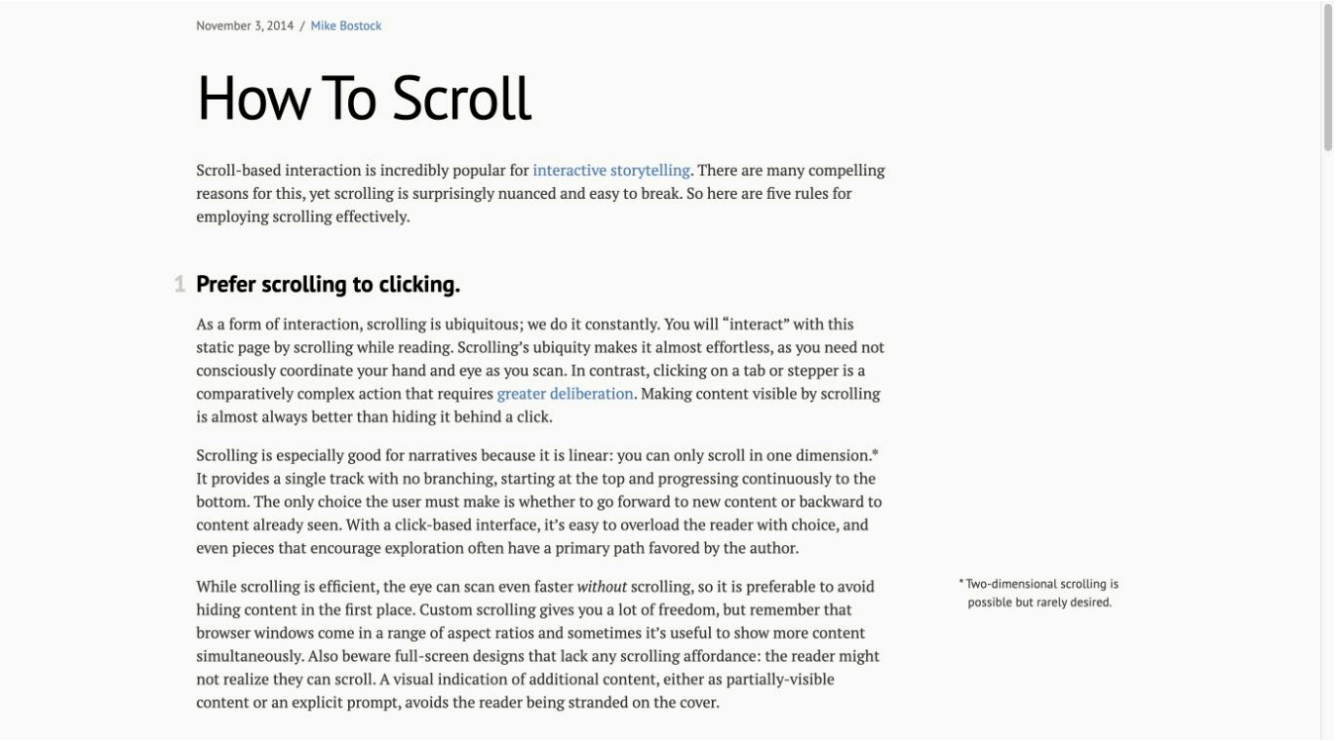
Maggie Appleton 원문 발표 그림 옆에 그때 한 말이 붙어 있어 읽던 자리를 놓치지 않음.



Robin Rendle 원문 검은 면에는 짧은 문장, 흰 면에는 그림 하나. 어디를 볼지 분명함.



Frank Chimero 원문 넓은 왼쪽 여백 덕분에 오른쪽 글과 그림에 시선이 머묾.



November 3, 2014 / Mike Bostock

How To Scroll

Scroll-based interaction is incredibly popular for interactive storytelling. There are many compelling reasons for this, yet scrolling is surprisingly nuanced and easy to break. So here are five rules for employing scrolling effectively.

1 Prefer scrolling to clicking.

As a form of interaction, scrolling is ubiquitous; we do it constantly. You will "interact" with this static page by scrolling while reading. Scrolling's ubiquity makes it almost effortless, as you need not consciously coordinate your hand and eye as you scan. In contrast, clicking on a tab or stepper is a comparatively complex action that requires greater deliberation. Making content visible by scrolling is almost always better than hiding it behind a click.

Scrolling is especially good for narratives because it is linear: you can only scroll in one dimension.* It provides a single track with no branching, starting at the top and progressing continuously to the bottom. The only choice the user must make is whether to go forward to new content or backward to content already seen. With a click-based interface, it's easy to overload the reader with choice, and even pieces that encourage exploration often have a primary path favored by the author.

While scrolling is efficient, the eye can scan even faster *without* scrolling, so it is preferable to avoid hiding content in the first place. Custom scrolling gives you a lot of freedom, but remember that browser windows come in a range of aspect ratios and sometimes it's useful to show more content simultaneously. Also beware full-screen designs that lack any scrolling affordance: the reader might not realize they can scroll. A visual indication of additional content, either as partially-visible content or an explicit prompt, avoids the reader being stranded on the cover.

* Two-dimensional scrolling is possible but rarely desired.

[Mike Bostock 원문](#) 주석이 오른쪽에 짧게 빠져 있어 읽던 문장으로 돌아가기 쉬움.

사례 검색과 비교 기준

처음에는 한 파일로 작동하는 사례를 찾았다. 두 번째에는 기준이 될 실물을 하나 짚었고, 세 번째에는 내가 열어 본 세 글의 주소를 건넸다.

Prompt 3. 단일 HTML 사례

> CSS와 JavaScript를 HTML 파일 하나에 담아 만든 프로젝트 세 가지를 찾아 줘.
> 설치 없이 브라우저에서 열리고, 읽는 사람이 직접 조작해 결과를 확인할 수 있는 사례로 골라 줘.
> 각 사례의 이름, 핵심 동작, 한 파일로 만들었는지 확인할 수 있는 근거와 실제로 열리는 원문 링크를 적어 줘.
> 단일 HTML인지 확인되지 않은 사례는 따로 구분하고, 조건을 충족하는 사례가 적으면 개수를 억지로 채우지 말아 줘.

Prompt 4. 기준 사례 비교

> MSCHF처럼 익숙한 규칙이나 시스템을 직접 체험하게 만드는 수학 프로젝트 세 가지를 찾아 줘.
> 공식 설명을 읽는 데서 끝나지 않고, 사용자의 선택이나 조작에 따라 결과가 달라지는 사례로 골라 줘.
> 각 사례에서 사용자가 하는 행동, 드러나는 수학적 규칙, MSCHF와 닮은 구조를 구분해서 설명해 줘.
> 실제로 열리는 원문 링크를 붙이고, 겉모습만 비슷한 경우와 상호작용의 구조까지 비슷한 경우를 나눠 줘.

Prompt 5. 사례 조합

> 아래 세 사례를 비교해서, 동료에게 AI 활용법을 소개하는 HTML 발표 페이지에 적용할 구성 세 가지를 제안해 줘.
> https://simonwillison.net/2023/Aug/6/annotated-presentations/
> https://maggieappleton.com/forest-talk
> https://bost.ocks.org/mike/scroll
> 발표자가 10분 동안 넘겨 보고, 발표 뒤에는 동료가 같은 파일을 혼자 읽으며 요청문을 복사할 수 있어야 해.
> 각 제안에 참고한 원문의 링크, 가져올 방식, 이 발표에 맞게 바꿀 점을 적어 줘.
> 세 사례의 설명 방식과 이동 방식을 비교하되, 기능이 겹치면 한 가지로 합치고 원문에서 확인되지 않은 동작은 추정이라고 표시해 줘.

Prompt 6. 결과 확인

> 기획서와 HTML 파일을 줄게. Chrome에 해당 HTML을 열어 두었어.
> 기획서의 완료 기준을 실제 화면에서 조작하며 확인하고, 기대 결과와 실제 결과를 비교해 줘.
> 문제가 있으면 재현 환경과 조작 순서, 기대 결과, 실제 결과를 적어 줘.
> 확인한 것, 추론한 것, 확인하지 못한 것을 구분하고 만든 AI의 통과 보고만으로 통과 처리하지 말아 줘.

Q&A